

Operating Systems: Three Easy Pieces — Notes

Luke
Independent Study

09 May 2026
Version 0.1

— Abstract —

Personal reading notes on *Operating Systems: Three Easy Pieces* by Remzi and Andrea Arpaci-Dusseau. Covers the three core themes of the text: virtualization, concurrency, and persistence. Written as part of a self-directed systems curriculum toward graduate CS study and ML/RL engineering work.

The book has three major themes:

- Virtualization
- Concurrency
- Persistence

1 Virtualization

Virtualization - The Operating System takes a physical resource (such as the processor, memory, or disk) and transforms it into a more general, powerful, and easy-to-use virtual form of itself. Thus the operating system is sometimes referred to as a *virtual machine*.

Crux: How to Virtualize Resources

- What mechanisms and policies are implemented by the OS to attain virtualization?
- How does the OS do so efficiently?
- What hardware support is needed?

A typical operating system provides a set of a few hundred system calls that are available to applications in the form of an API. We could say that the operating system provides a *standard library* to applications.

Because the OS allows for multiple programs to run programs, access memory and devices, the OS is sometimes called the *resource manager*.

Turning a CPU (or a small set of them) into a seemingly infinite number of CPUs and thus allowing many programs to seemingly run at once is **virtualizing the CPU**.

Open Question - If two programs want to run at a particular time, which *should* run? This is answered by a **policy** of the operating system.

When running programs, each can have its own private virtual address space, which the OS maps to physical memory on the machine. A memory reference in one running program does not affect the address space of other processes (or the OS itself).

Note: Read about Address Space Layout Randomization later (ASLR).

2 Concurrency

Volatile - Keyword in C, tells the compiler not to optimize reads/writes to that variable, every access must go directly to memory, never a cached register value. (Generally atomic is used for multi-threaded code)

Mutex - Mutual Exclusion Lock, a synchronization primitive that ensures only one thread can access a shared resource at a time.

atoi - Converts a command line string argument into an int, generally *strtol* is preferred in modern code implementations.

Crux: How to build Correct Concurrent Programs

When there are many concurrently executing threads within the same memory space, how can we build a correctly working program? What primitives are needed from the OS? What mechanisms should be provided by the hardware? How can we use them to solve the problems of concurrency?

Atomic Execution - All happening at once.

3 Persistence

Persistence - The storage of data persisting through power off or system crashes.

This problem is typically solved using an I/O system like Solid State Drives (SSDs).

Crux: How to Store Data Persistently

The file system is the part of the OS in charge of managing persistent data. What techniques are needed to do so correctly? What mechanisms and policies are required to do so with high performance? How is reliability achieved, in the face of failures in hardware and software?

In order to write to a disk the file system must first do a number of tasks:

Figure out where on the disk the new data will reside. Keep track of it in various structures that the file system maintains. Doing so requires issuing I/O requests to the underlying storage device, to either read existing structures or write to them.

Journaling / Copy-on-Write - Writing protocols that carefully order writes to the disk to ensure that if a failure occurs during the write sequence, the system can recover to a reasonable state afterwards.